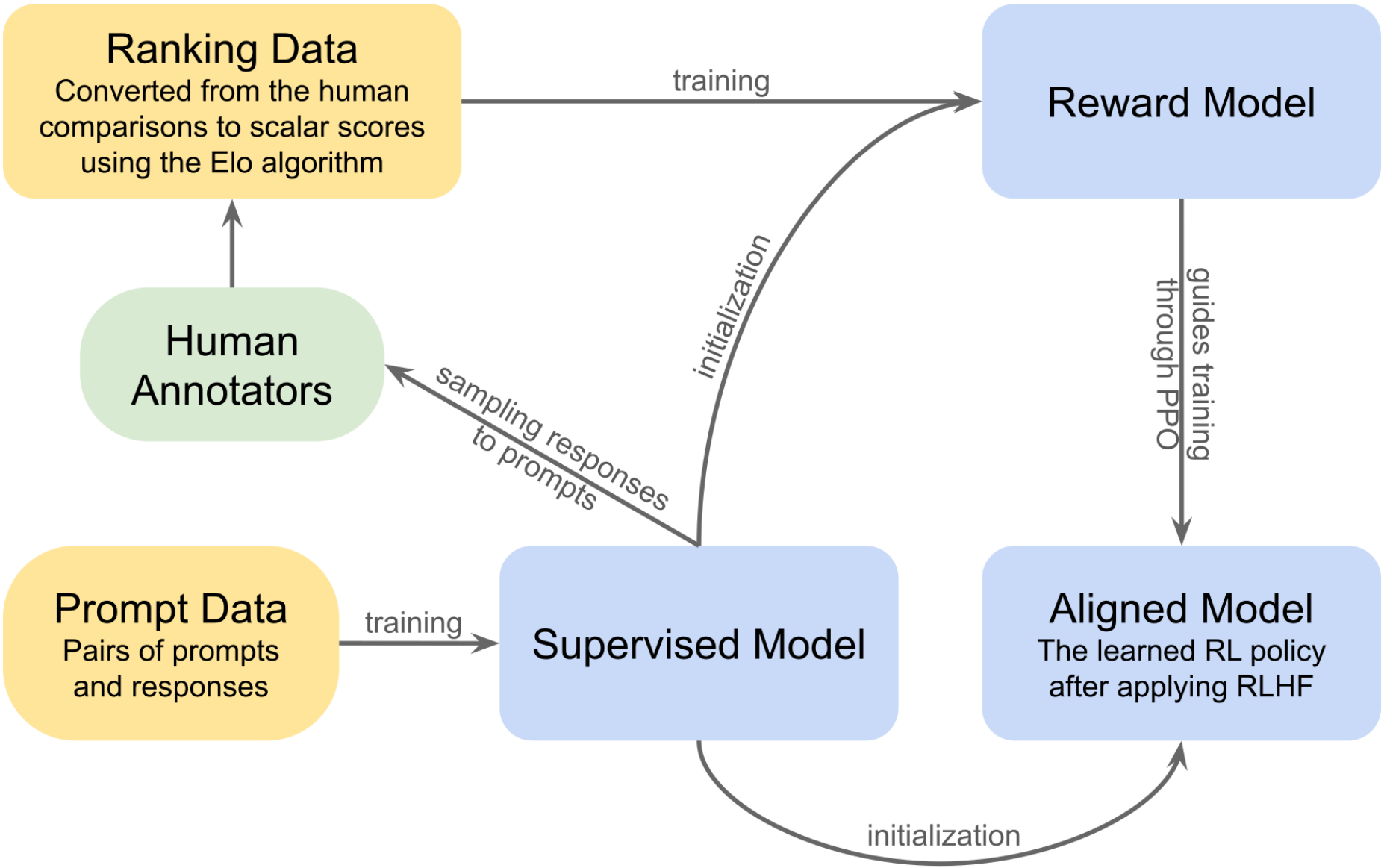
HybridFlow

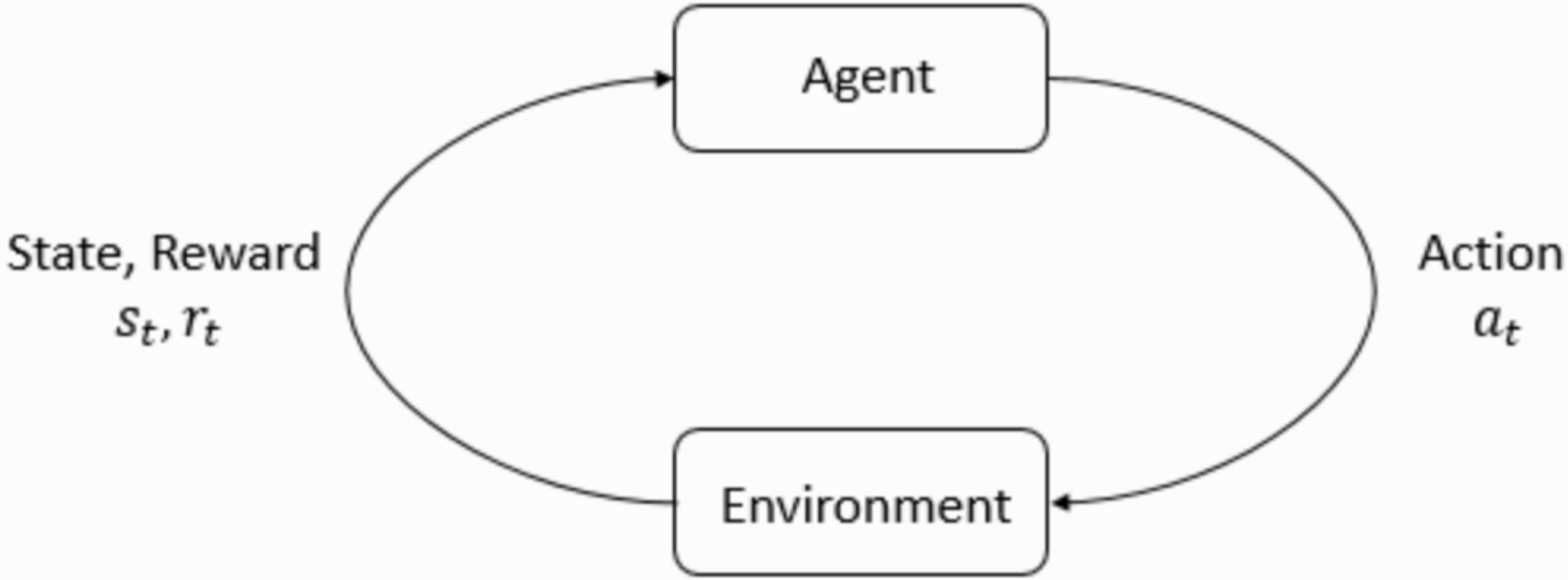
Danyang Zhuo

Duke University



From Wikipedia

Key Concepts and Terminology



Agent-environment interaction loop.

From “OpenAI spinning up”

The RL Problem

Whatever the choice of return measure (whether infinite-horizon discounted, or finite-horizon undiscounted), and whatever the choice of policy, the goal in RL is to select a policy which maximizes **expected return** when the agent acts according to it.

To talk about expected return, we first have to talk about probability distributions over trajectories.

Let's suppose that both the environment transitions and the policy are stochastic. In this case, the probability of a T -step trajectory is:

$$P(\tau|\pi) = \rho_0(s_0) \prod_{t=0}^{T-1} P(s_{t+1}|s_t, a_t)\pi(a_t|s_t).$$

The expected return (for whichever measure), denoted by $J(\pi)$, is then:

$$J(\pi) = \int_{\tau} P(\tau|\pi)R(\tau) = \underset{\tau\sim\pi}{\mathrm{E}}\left[R(\tau)\right].$$

The central optimization problem in RL can then be expressed by

$$\pi^* = \arg\max_{\pi} J(\pi),$$

with π^* being the **optimal policy**.

From "OpenAI spinning up"

Value Functions

It's often useful to know the **value** of a state, or state-action pair. By value, we mean the expected return if you start in that state or state-action pair, and then act according to a particular policy forever after. **Value functions** are used, one way or another, in almost every RL algorithm.

1. The **On-Policy Value Function**, $V^{\pi}(s)$, which gives the expected return if you start in state s and always act according to policy π:

$$V^{\pi}(s) = \underset{\tau \sim \pi}{\mathrm{E}} \left[R(\tau) \,|s_0 = s \right]$$

2. The **On-Policy Action-Value Function**, $Q^{\pi}(s, a)$, which gives the expected return if you start in state s, take an arbitrary action a (which may not have come from the policy), and then forever after act according to policy π:

$$Q^{\pi}(s, a) = \underset{\tau \sim \pi}{\mathrm{E}} \left[R(\tau) \,|s_0 = s, a_0 = a \right]$$

From "OpenAI spinning up"

Advantage Functions

Sometimes in RL, we don't need to describe how good an action is in an absolute sense, but only how much better it is than others on average. That is to say, we want to know the relative **advantage** of that action. We make this concept precise with the **advantage function.**

The advantage function $A^\pi(s,a)$ corresponding to a policy π describes how much better it is to take a specific action a in state s, over randomly selecting an action according to $\pi(\cdot|s)$, assuming you act according to π forever after. Mathematically, the advantage function is defined by

$$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s).$$

From "OpenAI spinning up"

Pseudocode

Algorithm 1 PPO-Clip

Input: initial policy parameters θ_0, initial value function parameters ϕ_0

for $k = 0, 1, 2, \ldots$ **do**

  Collect set of trajectories $\mathcal{D}_k = \{\tau_i\}$ by running policy $\pi_k = \pi(\theta_k)$ in the environment.

  Compute rewards-to-go $\hat{R}_t$.

  Compute advantage estimates, $\hat{A}_t$ (using any method of advantage estimation) based on the current value function V_{ϕ_k}.

  Update the policy by maximizing the PPO-Clip objective:

$$\theta_{k+1} = \arg\max_{\theta} \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^{T} \min\left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)} A^{\pi_{\theta_k}}(s_t, a_t), \quad g(\epsilon, A^{\pi_{\theta_k}}(s_t, a_t)) \right),$$

  typically via stochastic gradient ascent with Adam.

  Fit value function by regression on mean-squared error:

$$\phi_{k+1} = \arg\min_{\phi} \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^{T} \left(V_\phi(s_t) - \hat{R}_t \right)^2,$$

  typically via some gradient descent algorithm.

end for

From “OpenAI spinning up”

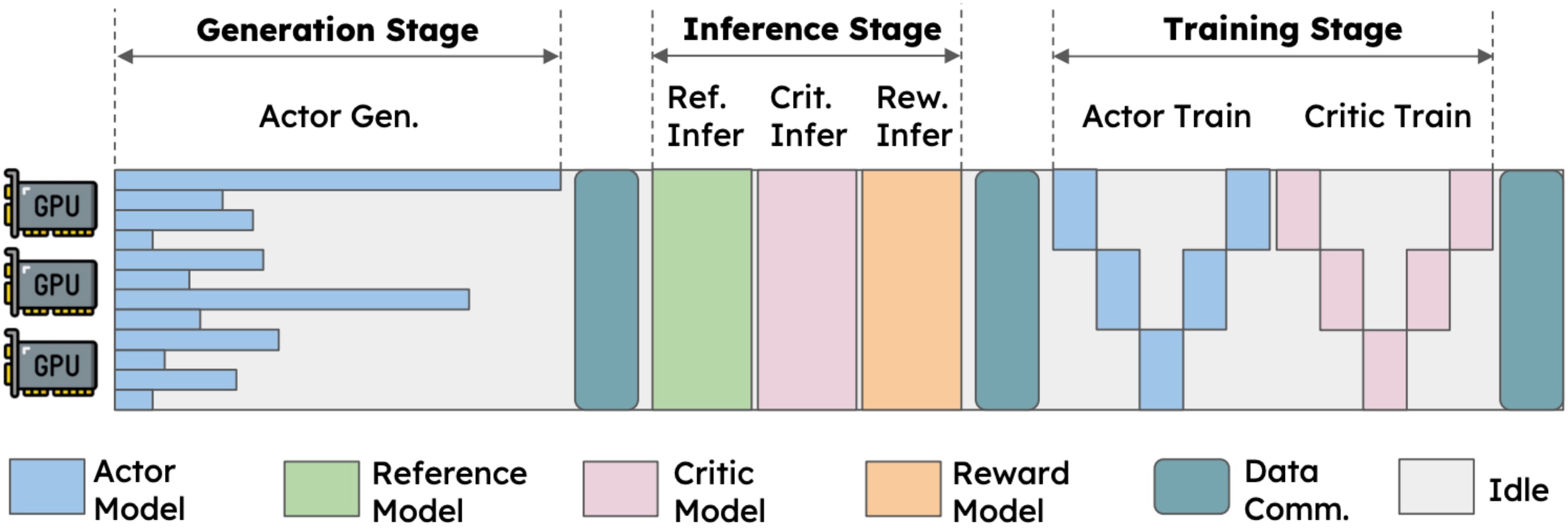


Figure 1: RLHF iteration breakdown of existing frameworks.

From “RLHFuse”

Cost of RLHF using PPO

- Actor generating several sequences for a set of prompts
  - Prefill + decoding
- Reference model generate the probabilities for the generated sequences
  - Similar as prefill
- Critic and reward models evaluates the generated sequences
  - Similar as prefill
- Update critic model and actor model using backpropagation

HybridFlow

- Hybrid Programming Model
  - In contrast with single controller or multi controller RL
- 3D-Hybrid Engine
- Auto GPU allocation and compute placement

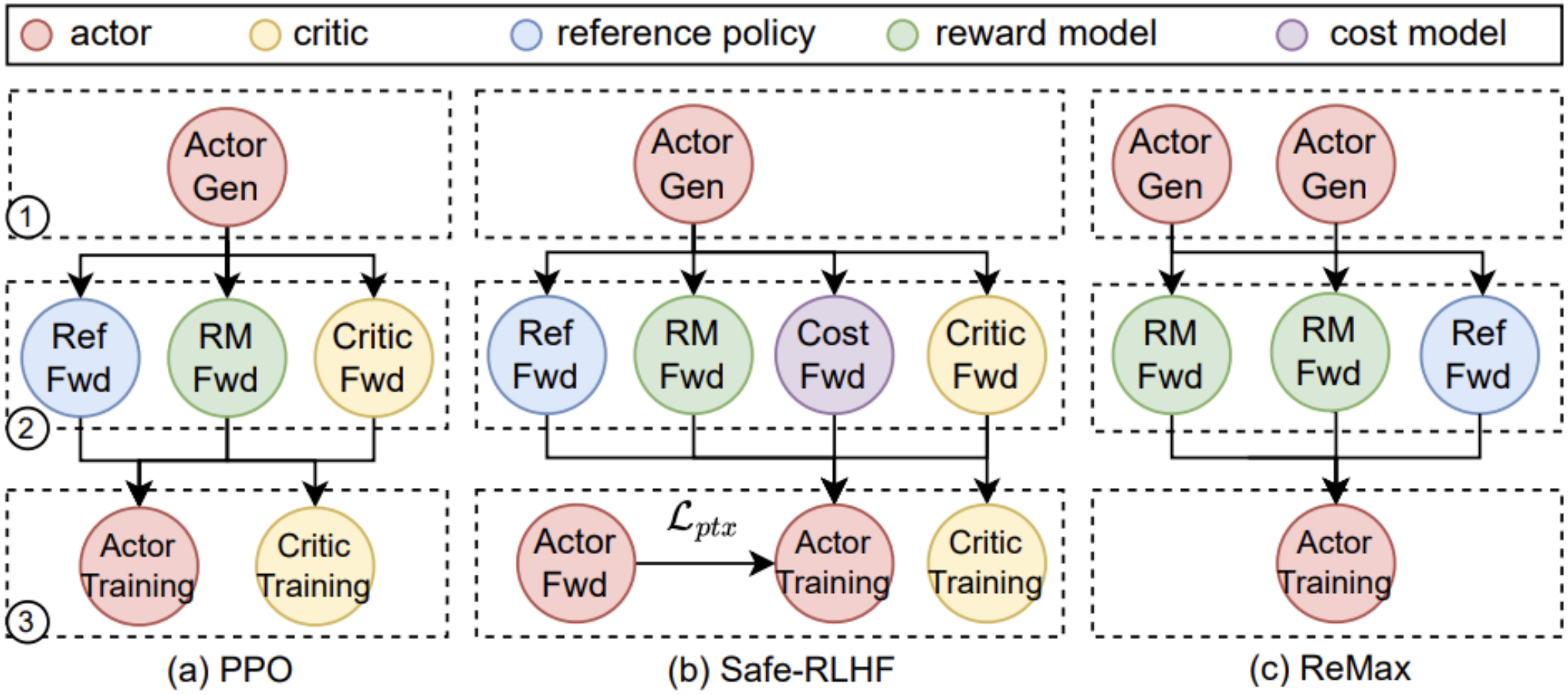
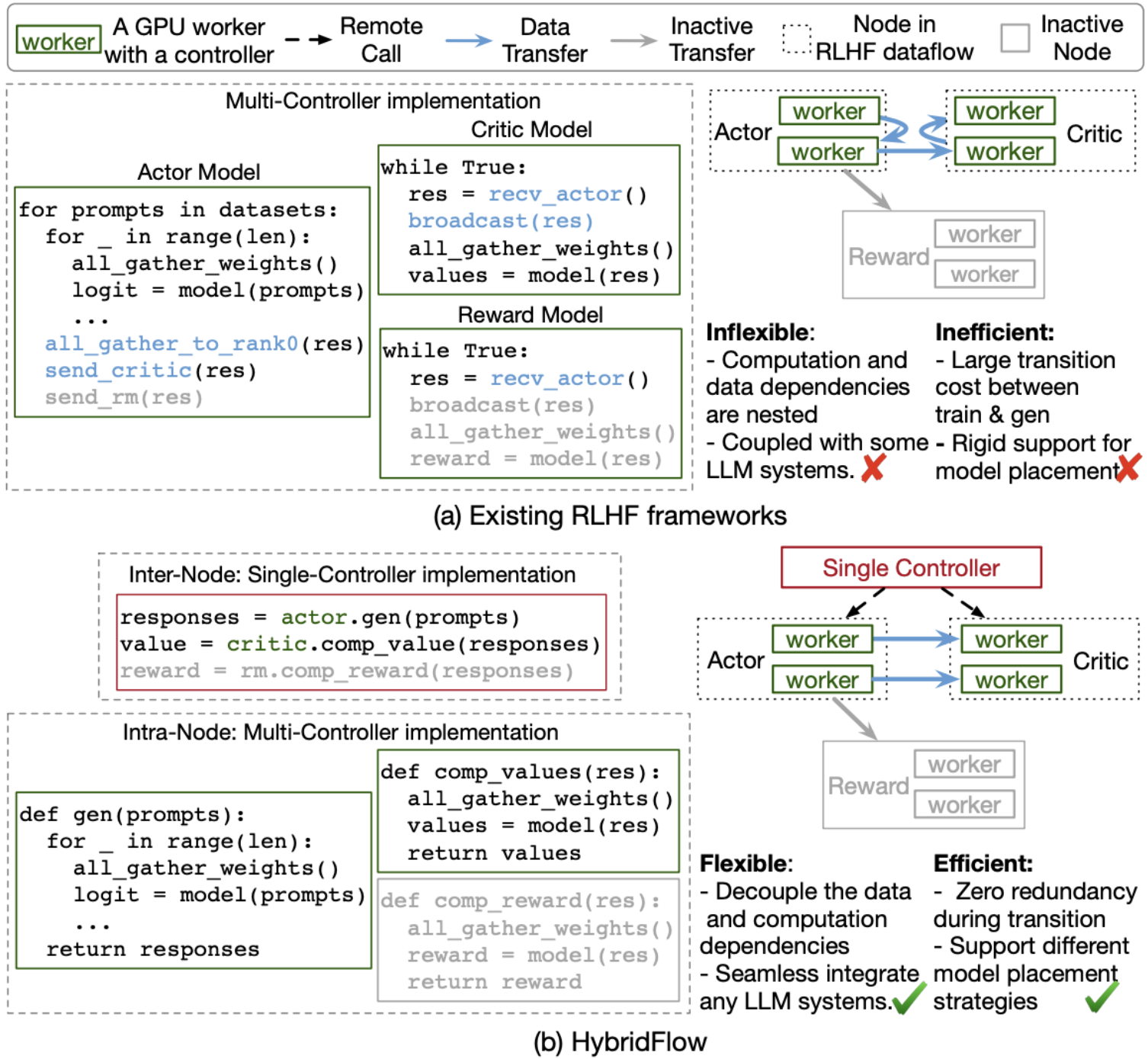


Figure 1. Dataflow graph of 3 RLHF algorithms [19, 43, 55]. Stage ①, ②, ③ represent Generation, Preparation, and Training, respectively.

Two types of RL systems

- Single controller
  - Single process sending control messages to all workers
  - Easy to develop: the controller has global view of resources and is the only controller
  - Dispatch overheads is large.

- Multi controllers
  - Each worker can the same or different code and communicate via network
  - Dispatch overheads is low
  - Communication is programmed explicitly


worker
A GPU worker with a controller
Remote Call
Data Transfer
Inactive Transfer
Node in RLHF dataflow
Inactive Node
Multi-Controller implementation
Actor Model
for prompts in datasets:
for _ in range(len):
all_gather_weights()
logit = model(prompts)
...
all_gather_to_rank0(res)
send_critic(res)
send_rm(res)
Critic Model
while True:
res = recv_actor()
broadcast(res)
all_gather_weights()
values = model(res)
Reward Model
while True:
res = recv_actor()
broadcast(res)
all_gather_weights()
reward = model(res)
Actor
worker
worker
worker
worker
Critic
Reward
worker
worker
Inflexible:
- Computation and data dependencies are nested
- Coupled with some LLM systems.
Inefficient:
- Large transition cost between train & gen
- Rigid support for model placement
(a) Existing RLHF frameworks
Inter-Node: Single-Controller implementation
responses = actor.gen(prompts)
value = critic.comp_value(responses)
reward = rm.comp_reward(responses)
Single Controller
Actor
worker
worker
worker
worker
Critic
Reward
worker
worker
Intra-Node: Multi-Controller implementation
def gen(prompts):
for _ in range(len):
all_gather_weights()
logit = model(prompts)
...
return responses
def comp_values(res):
all_gather_weights()
values = model(res)
return values
def comp_reward(res):
all_gather_weights()
reward = model(res)
return reward
Flexible:
- Decouple the data and computation dependencies
- Seamless integrate any LLM systems.
Efficient:
- Zero redundancy during transition
- Support different model placement strategies
(b) HybridFlow

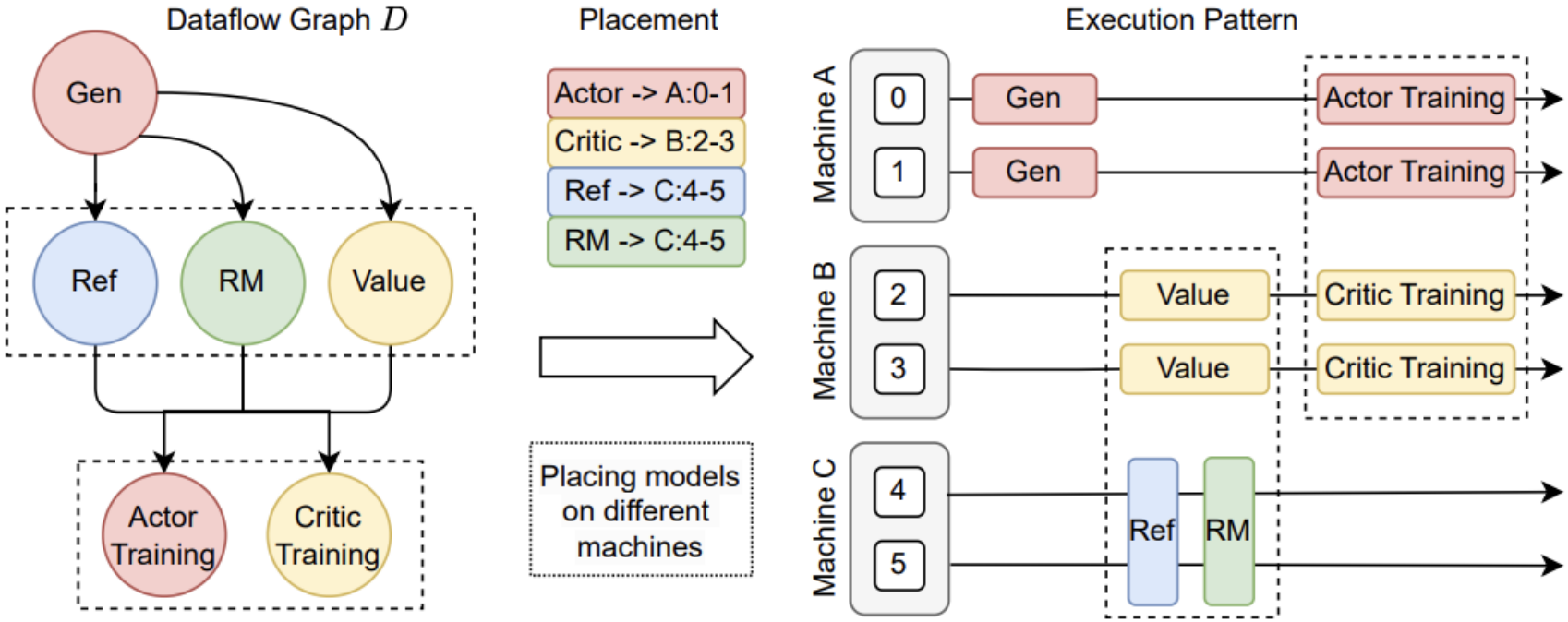
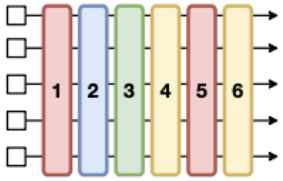
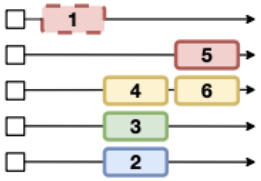
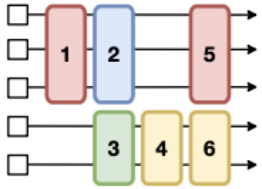


Figure 3. Dataflow execution given a model placement plan. Blocks with numbers represent GPUs. In dashed boxes, the models are placed on different sets of devices and can be concurrently computed. Reference model (blue) and reward model (green) are colocated on the same set of GPUs and executed sequentially.

Table 1. Comparison of RLHF frameworks. Figures illustrate execution of one PPO iteration. Numbers 1-6 represent response generation, reward model inference, reference model inference, critic inference, actor training, and critic training, respectively.

RLHF system	DeepSpeed-Chat	OpenRLHF	NeMo-Aligner	**HybridFlow**
Parallelism	Training: ZeRO Generation:TP	Training: ZeRO Generation:TP	3D Parallelism for both training and generation	**Training: 3D, ZeRO, FSDP** **Generation: 3D Parallelism**
Actor weights in training & generation	Model resharding from ZeRO to TP	Using two copies of actor weights for the two stages	Using identical model partition in two stages (shared weights)	**Zero-redundancy** **model resharding**
Model Placement	Colocate all models on the same set of devices	Each model placed on separate devices	Actor/Ref colocated on some GPUs Critic/RM colocated on other GPUs	**Support various** **model placement**
Execution Pattern Actor GPU Process Critic Reward model Reference Policy				**Support various** **execution patterns**

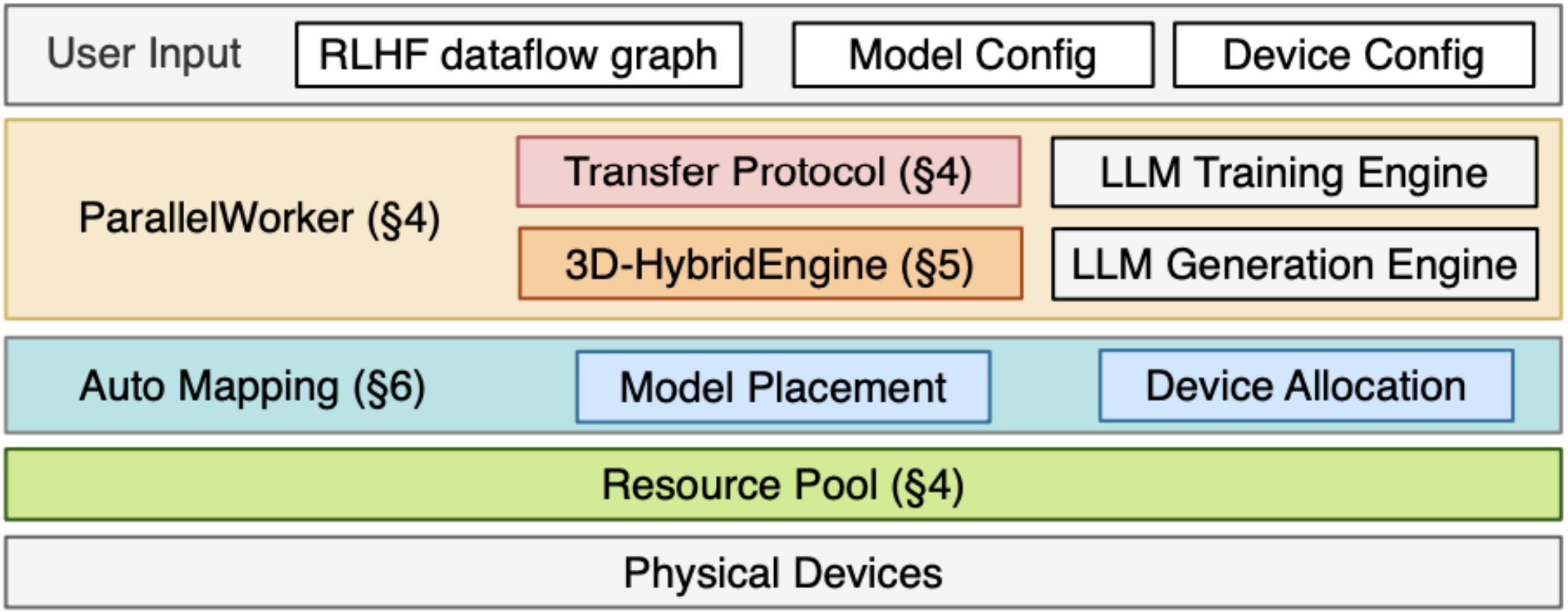


Figure 4. Architecture of HybridFlow.

```python
class ActorWorker(3DParallelWorker):
    # An example of distributed computation function
    @register(transfer_mode=3D_PROTO)
    def update_actor(self, prompts: DataProto):
        ...
# Allocate devices for a ResourcePool
resource_pool = ResourcePool([n_gpus_per_machine] * n_machines)
# Map the model to allocated devices and init model
actor_model = ActorWorker(actor_config, resource_pool)
```

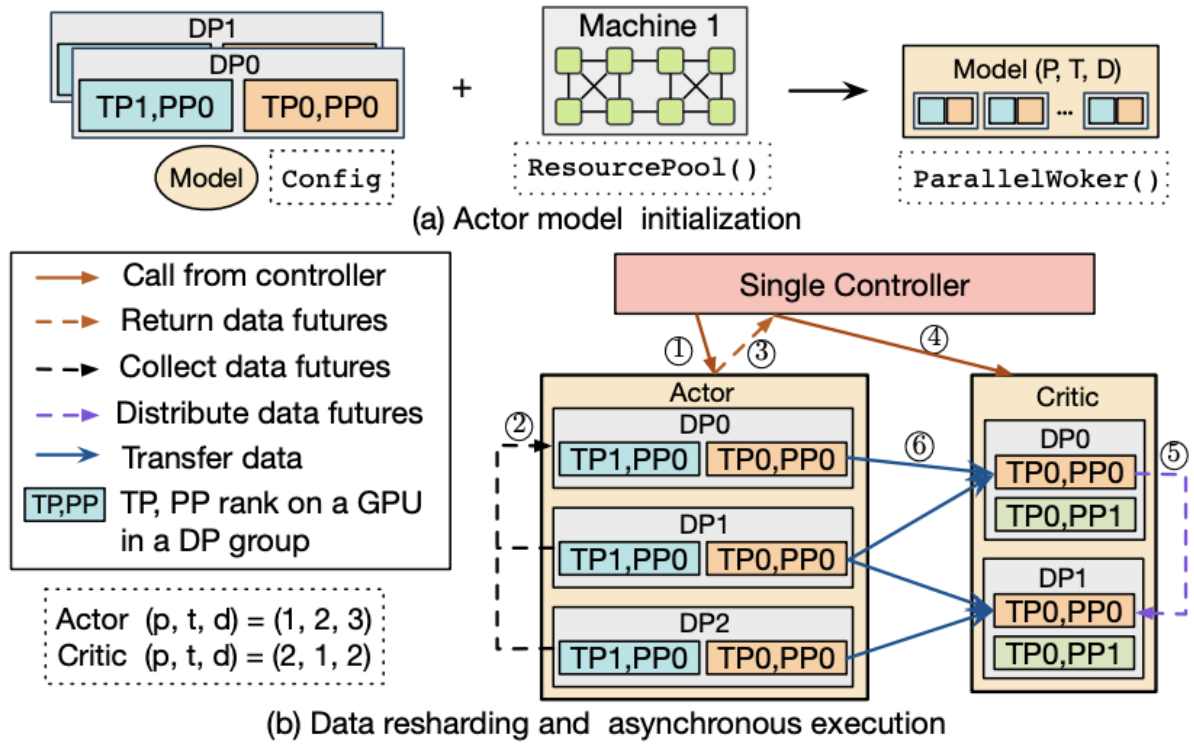


Figure 5. An illustration of hierarchical APIs. (a) Model with 3D parallel configuration, resource allocation, and `3DParallelWorker` initialization. (b) Asynchronous data resharding between two models with collect and distribute functions in `3D_PROTO`.

```
# Initialize cost model by reusing the RewardWorker
cost = RewardWorker(cost_config, resource_pool)
...  # omit other models initialization
algo_type = "Safe-RLHF" # specify different RLHF numerical computation.
# Examples of PPO and Safe-RLHF
for (prompts, pretrain_batch) in dataloader:
    # Stage 1: Generate responses
    batch = actor.generate_sequences(prompts)
    batch = actor.generate_sequences(prompts, do_sample=False)
    # Stage 2: Prepare experience
    batch = critic.compute_values(batch)
    batch = reference.compute_log_prob(batch
    batch = reward.compute_reward(batch)
    batch = cost.compute_cost(batch)
    batch = compute_advantages(batch, algo_type)
    # Stage 3: Actor and critic training
    critic_metrics = critic.update_critic(batch, loss_func=algo_type)
    pretrain_loss = actor.compute_loss(pretrain_batch)
    batch["pretrain_loss"] = pretrain_loss
    actor_metrics = actor.update_actor(batch, loss_func=algo_type)
```

is added for ReMax

Not necessary in ReMax

is added for Safe-RLHF

Figure 6. Implementation of PPO [55], ReMax [43], and Safe-RLHF [19]. Users can adapt to different RLHF algorithms by simply adding or deleting a few lines of code.

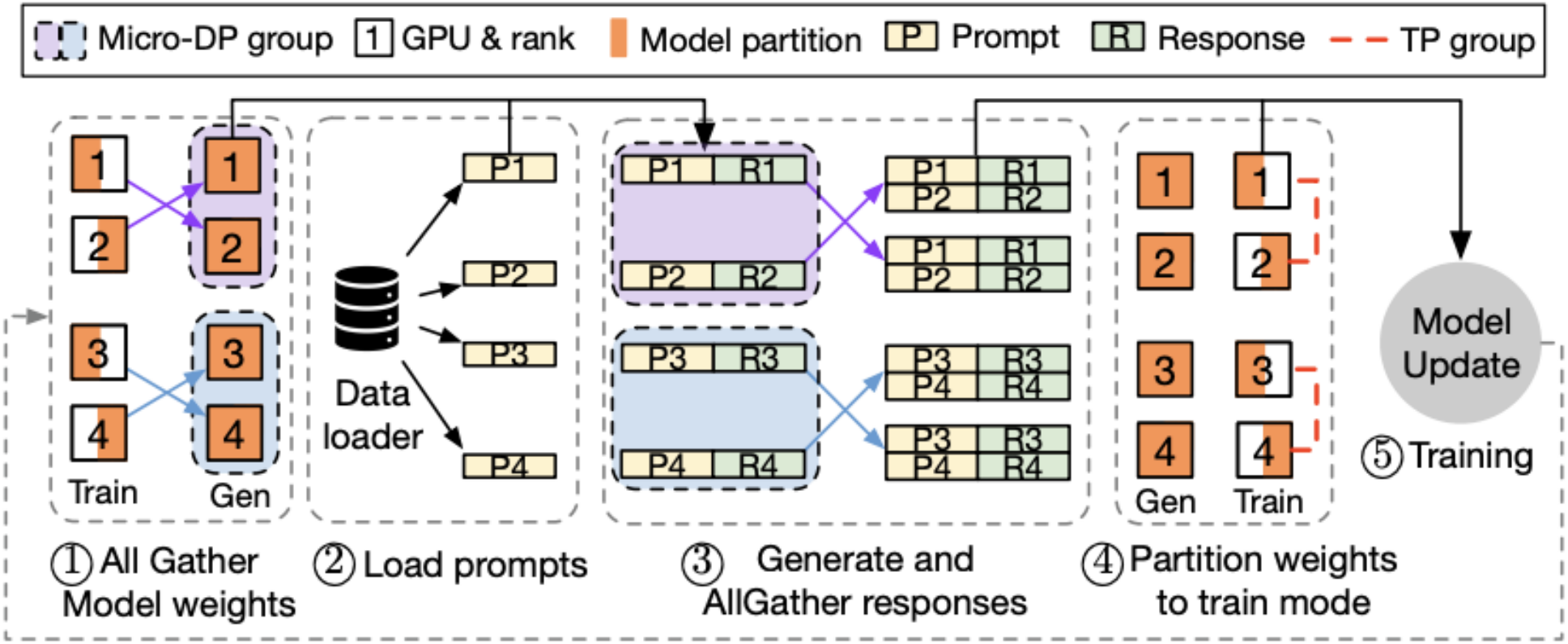


Figure 7. 3D-HybridEngine workflow in one RLHF iteration. 4 GPUs are used for actor training and generation. 1-2-2 (p-t-d) parallel groups are used in training and 1-1-2-2 (p_g-t_g-d_g-d) parallel groups are used in generation.

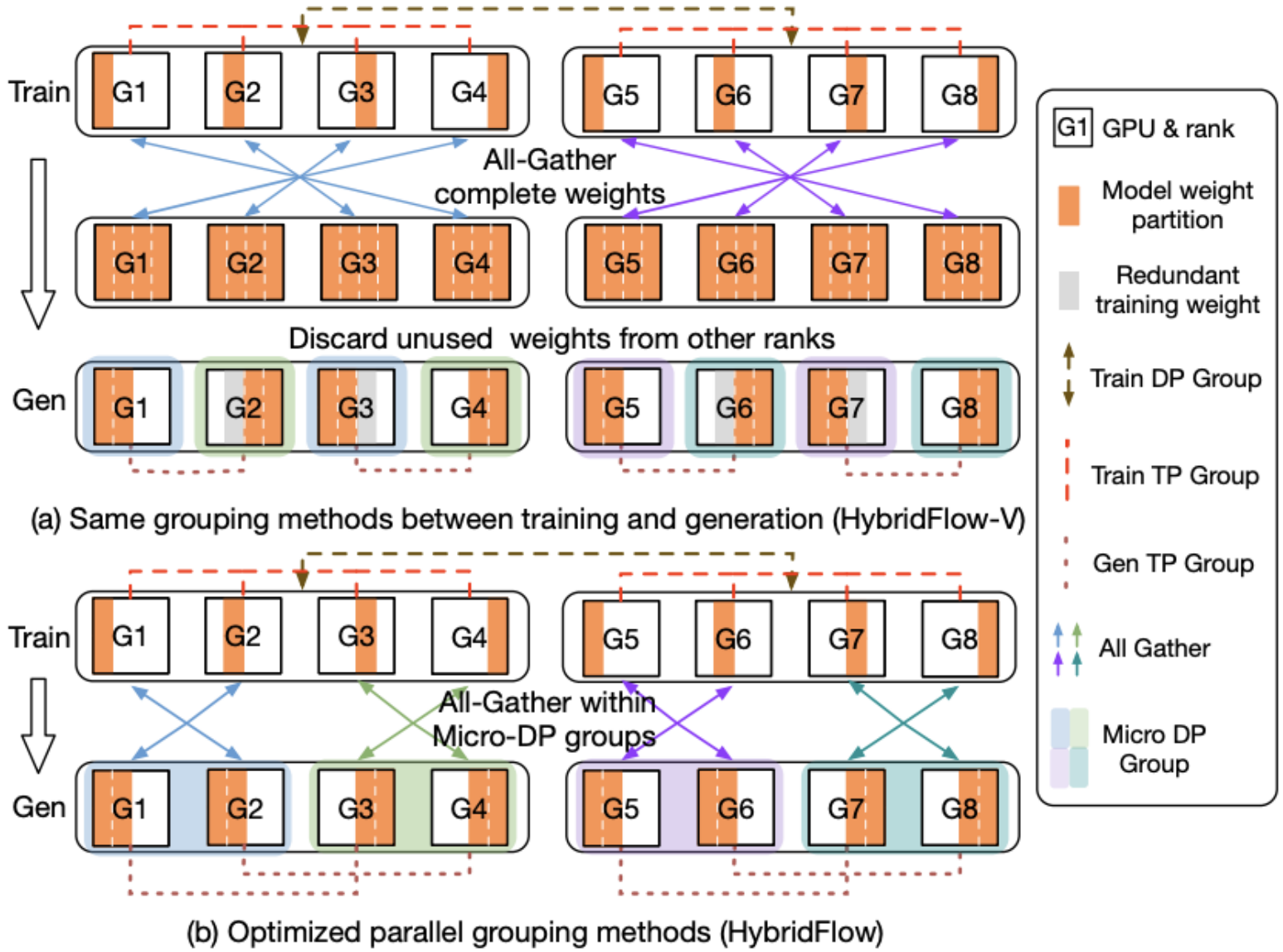


Figure 8. Model weights resharding. 2 machines each with 4 GPUs are used for actor training and generation.

Auto GPU allocation and compute placement

- Map from RLHF dataflow to a cluster of GPUs, minimizing end-to-end latency

- Search using the nested loops
  - Loop over placement plan (i.e., whether reward and critic are located on the same GPU set)
  - Loop over resource assignment to each placement (i.e., how many GPUs are assigned to reward + critic)
  - Estimate cost

Algorithm 1 Device Mapping for an RLHF Dataflow

Input: RLHF dataflow graph D, LLMs in RLHF dataflow $L=[l_1, l_2, \ldots, l_k]$, workload W of LLMs in RLHF dataflow, total # of GPUs N, memory capacity per GPU Q
Output: device mapping of models in RLHF dataflow
$\mathcal{P} \leftarrow$ get_placements(D, L, N)
$C^* \leftarrow \infty$
$best_mapping \leftarrow \emptyset$
for all $plm \in \mathcal{P}$ **do**
  $C_{plm} \leftarrow \infty$
  $best_plm_alloc \leftarrow \emptyset$
  $A_{min} \leftarrow$ get_min_alloc(plm, Q, N)
  for all $A \in$ enum_alloc(N, A_{min}) **do**
    $\widehat{L} \leftarrow []$
    for all set $\in plm$ **do**
      for all $l \in$ set **do**
        $\widehat{l} \leftarrow$ auto_parallel(A, A_{min}, l, W)
        $\widehat{L}$.append$(\widehat{l})$
    plm.update$(\widehat{L})$
    $C_{alloc} \leftarrow$ d_cost(D, plm, W)
    if $C_{alloc} < C_{plm}$ **then**
      $C_{plm} \leftarrow C_{alloc}$
      $best_plm_alloc \leftarrow (plm, A)$
  if $C_{plm} < C^*$ **then**
    $C^* \leftarrow C_{plm}$
    $best_mapping \leftarrow best_plm_alloc$
return $best_mapping$
Procedure d_cost(D, plm, W):
  $s \leftarrow$ number of stages in D
  $c \leftarrow [0] \times s$ *// Initialize latency for each stage to 0*
  for all set $\in plm$ **do**
    $c_g \leftarrow [0] \times s$
    for all $i \in \{0, \ldots, s-1\}$ **do**
      for all $\widehat{l} \in$ set **do**
        $c_g[i] \leftarrow c_g[i] + \text{simu}(\widehat{l}, W[i])$
      $c[i] \leftarrow max\{c[i], c_g[i]\}$
  return sum(c)

Rollout length issues

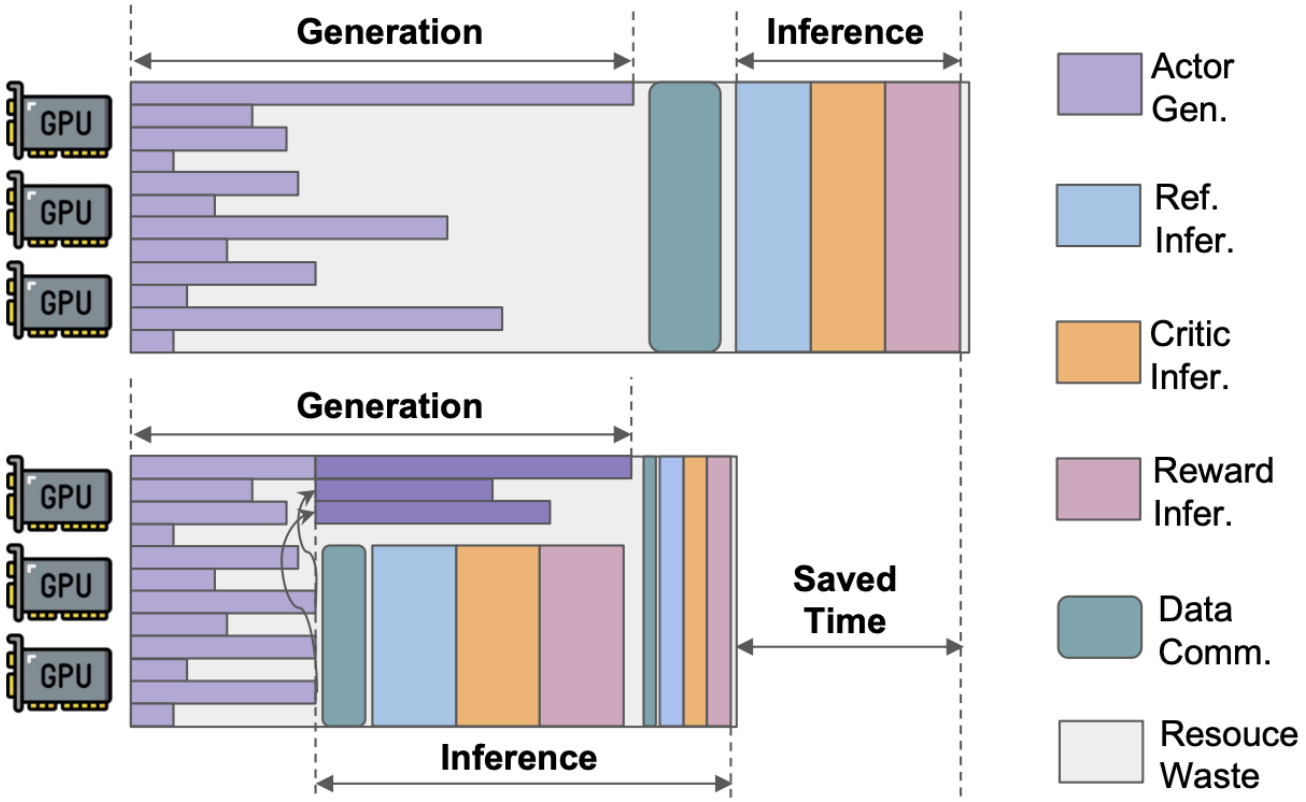
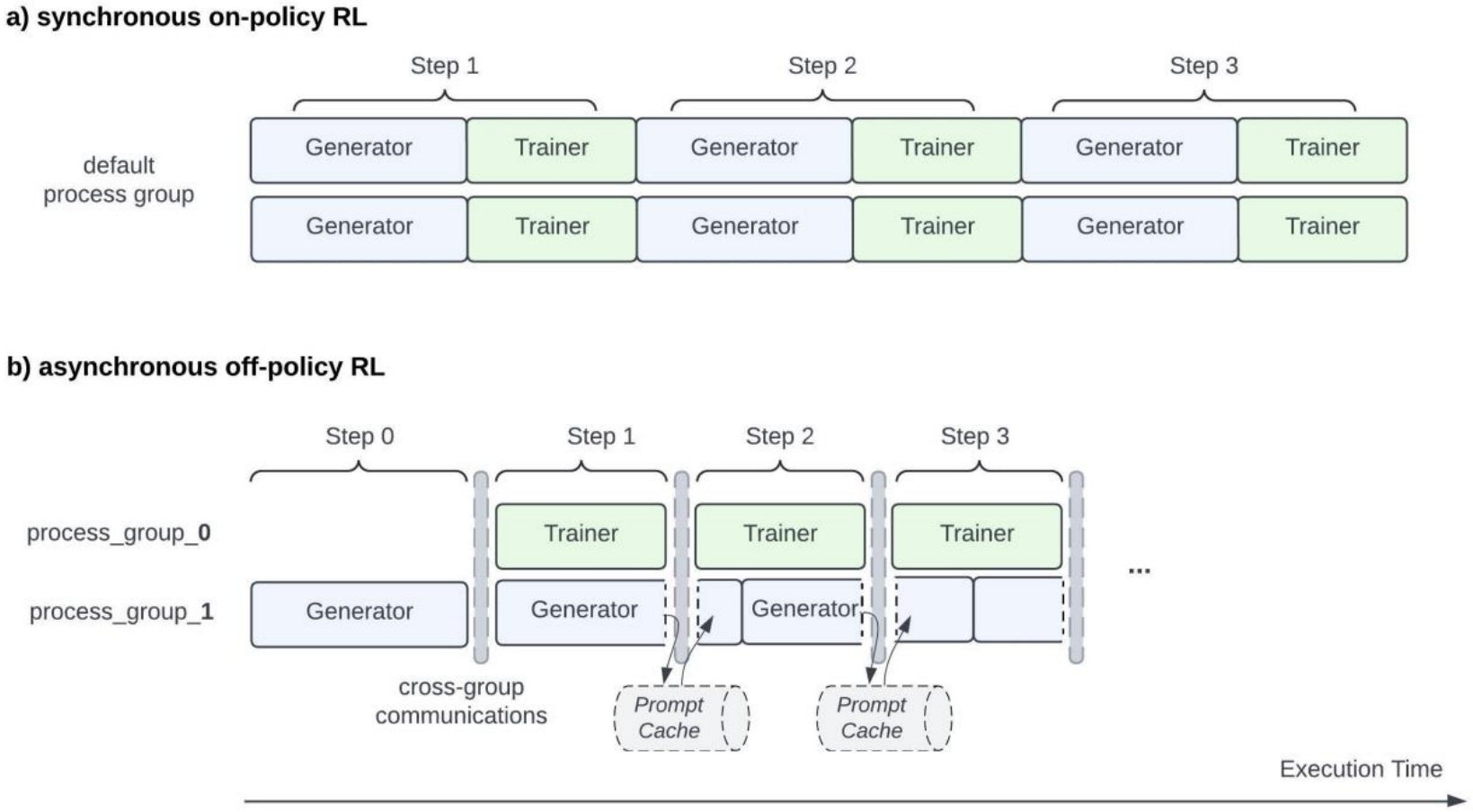


Figure 5: The timeline of serial (top) and fused (bottom) execution of generation and inference stages.

From “RLHFuse”

Rollout length issues



From “LlamaRL”

DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning

DeepSeek-AI

research@deepseek.com

Pseudocode

Algorithm 1 PPO-Clip

Input: initial policy parameters θ_0, initial value function parameters ϕ_0

for $k = 0, 1, 2, \ldots$ **do**

  Collect set of trajectories $\mathcal{D}_k = \{\tau_i\}$ by running policy $\pi_k = \pi(\theta_k)$ in the environment.

  Compute rewards-to-go $\hat{R}_t$.

  Compute advantage estimates, $\hat{A}_t$ (using any method of advantage estimation) based on the current value function V_{ϕ_k}.

  Update the policy by maximizing the PPO-Clip objective:

$$\theta_{k+1} = \arg\max_{\theta} \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^{T} \min\left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)} A^{\pi_{\theta_k}}(s_t, a_t), \quad g(\epsilon, A^{\pi_{\theta_k}}(s_t, a_t)) \right),$$

  typically via stochastic gradient ascent with Adam.

  Fit value function by regression on mean-squared error:

$$\phi_{k+1} = \arg\min_{\phi} \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^{T} \left(V_\phi(s_t) - \hat{R}_t \right)^2,$$

  typically via some gradient descent algorithm.

end for

From “OpenAI spinning up”

For each question q, GRPO samples a group of outputs $\{o_1, o_2, \cdots, o_G\}$ from the old policy $\pi_{\theta_{old}}$ and then optimizes the policy model π_θ by maximizing the following objective:

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^{G} \sim \pi_{\theta_{old}}(O|q)]$$
$$\frac{1}{G}\sum_{i=1}^{G}\left(\min\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}A_i, \text{clip}\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1-\varepsilon, 1+\varepsilon\right)A_i\right) - \beta\mathbb{D}_{KL}\left(\pi_\theta||\pi_{ref}\right)\right), \tag{1}$$

$$\mathbb{D}_{KL}\left(\pi_\theta||\pi_{ref}\right) = \frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - \log\frac{\pi_{ref}(o_i|q)}{\pi_\theta(o_i|q)} - 1, \tag{2}$$

where π_{ref} is a reference policy, ε and β are hyper-parameters, and A_i is the advantage, computed using a group of rewards $\{r_1, r_2, \ldots, r_G\}$ corresponding to the outputs within each group:

$$A_i = \frac{r_i - mean(\{r_1, r_2, \cdots, r_G\})}{std(\{r_1, r_2, \cdots, r_G\})}. \tag{3}$$

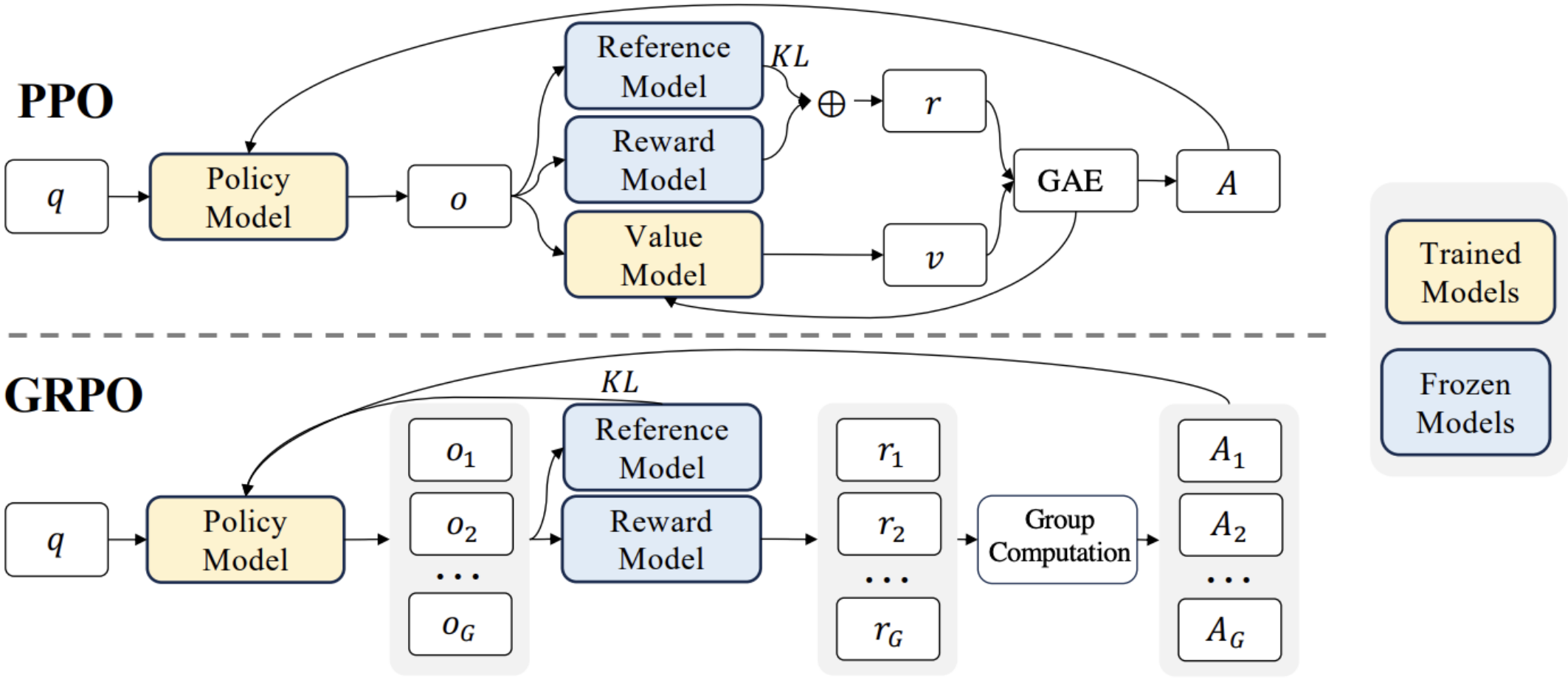


Figure 3 | Demonstration of PPO and our GRPO. GRPO foregoes the value model, instead estimating the advantages from group scores.

Table 1 | Template for DeepSeek-R1-Zero. prompt will be replaced with the specific reasoning question during training.

A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant first thinks about the reasoning process in the mind and then provides the user with the answer. The reasoning process and answer are enclosed within `<think>...</think>` and `<answer>...</answer>` tags, respectively, i.e., `<think>` reasoning process here `</think>` `<answer>` answer here `</answer>`. User: prompt. Assistant:

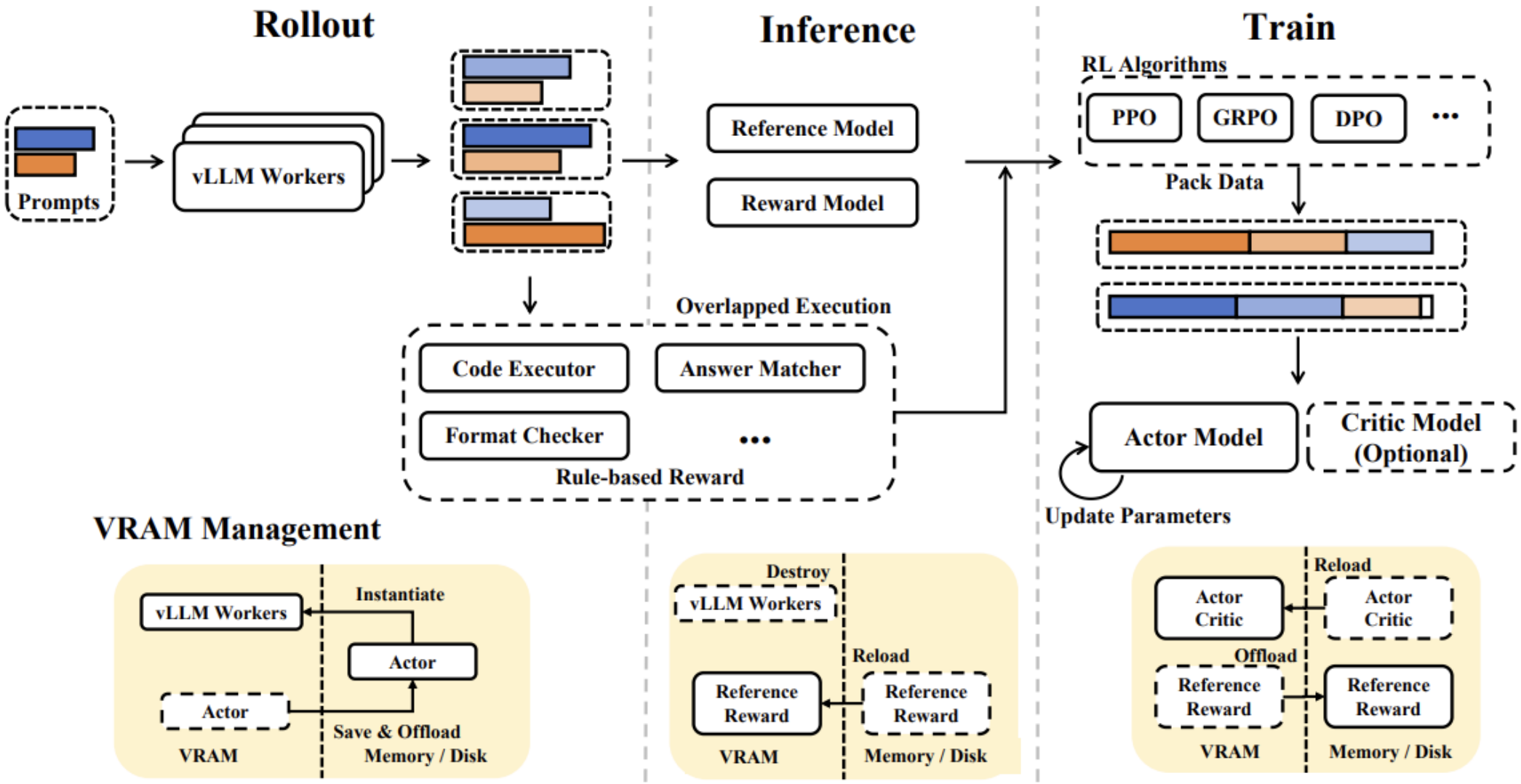


Figure 5 | Overview of our RL framework.

Table 2 | An interesting "aha moment" of an intermediate version of DeepSeek-R1-Zero. The model learns to rethink using an anthropomorphic tone. This is also an aha moment for us, allowing us to witness the power and beauty of reinforcement learning.

Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a - \sqrt{a + x}} = x$ is equal to
Response: `<think>` To solve the equation $\sqrt{a - \sqrt{a + x}} = x$, let's start by squaring both $\cdots$ $\left(\sqrt{a - \sqrt{a + x}}\right)^2 = x^2 \implies a - \sqrt{a + x} = x^2.$ Rearrange to isolate the inner square root term: $(a - x^2)^2 = a + x \implies a^2 - 2ax^2 + (x^2)^2 = a + x \implies x^4 - 2ax^2 - x + (a^2 - a) = 0$ $\ldots$ Wait, wait. Wait. That's an aha moment I can flag here. Let's reevaluate this step-by-step to identify if the correct sum can be $\cdots$ We started with the equation: $\sqrt{a - \sqrt{a + x}} = x$ First, let's square both sides: $a - \sqrt{a + x} = x^2 \implies \sqrt{a + x} = a - x^2$ Next, I could square both sides again, treating the equation: $\cdots$ $\ldots$